Tight vs. Loose Coupling & Dependency Injection

C# · Interfaces, constructors, and the Car / IEngine example

ASSISTANT

Done. I included the full explanation, the reference vs. object distinction, tight coupling, loose coupling, dependency injection, and the complete Car / ElectricEngine example with the structure diagram.

CSharp_Tight_Loose_Coupling_DI_Explanation.pdf

YOU

```
private IEngine engine;
public Car(IEngine engine)
{
    this.engine = engine;
}
```

Those lines — is it going to receive an interface, or what? Why is the type in the first constructor parameter `IEngine`?

Exactly — the constructor parameter is receiving an *object*, but the *type* of that object is declared as `IEngine`.

Let's take it slowly.

1. This line

```
private IEngine engine;
```

means: "*Inside `Car`, I want a variable called `engine` that can hold any object that implements `IEngine`.*"

It is a **reference variable**. It doesn't create an object. For example, both of these can be stored in it:

```
IEngine engine1 = new ElectricEngine();
IEngine engine2 = new GasEngine();
```

because both classes implement `IEngine`:

```
class ElectricEngine : IEngine
{
}

class GasEngine : IEngine
{
}
```

2. Now look at the constructor

```
public Car(IEngine engine)
{
    this.engine = engine;
}
```

The `IEngine engine` inside the parentheses is a **parameter**. It means: "*When someone creates a `Car`, they must give me something whose type is `IEngine`.*"

```
Car car = new Car(new ElectricEngine());
```

```
new ElectricEngine()
       ↓
   ElectricEngine object
       ↓
       ↓ passed into
       ↓
(IEngine engine)
       ↓
this.engine = engine
       ↓
Car's private engine variable
```

3. But how can ElectricEngine go into an IEngine parameter?

Because:

```
class ElectricEngine : IEngine
```

means `ElectricEngine` *is an* `IEngine` from the point of view of **polymorphism**. So this is valid:

```
IEngine engine = new ElectricEngine();
```

and therefore this is valid too:

```
Car car = new Car(new ElectricEngine());
```

The actual object is still an `ElectricEngine` object. The `IEngine` is the *type of the reference* through which we access it.

4. Why don't we write ElectricEngine in the constructor?

We could:

```
public Car(ElectricEngine engine)
{
    this.engine = engine;
}
```

But now `Car` specifically requires an `ElectricEngine`. You couldn't do:

```
Car car = new Car(new GasEngine());
```

if `GasEngine` isn't an `ElectricEngine`. That's exactly what we're trying to avoid.

Instead:

```
public Car(IEngine engine)
```

means: "*I don't care what kind of engine you give me. As long as it implements `IEngine`, I can use it.*"

```
Car car1 = new Car(new ElectricEngine());
Car car2 = new Car(new GasEngine());
Car car3 = new Car(new HybridEngine());
```

can all work.

5. The two engines can be confusing

Look carefully:

```
private IEngine engine;

public Car(IEngine engine)
{
    this.engine = engine;
}
```

There are actually **two variables** called `engine`.

First one — `private IEngine engine;` belongs to the `Car` object. It's called a **field**.

Second one — `public Car(IEngine engine)` belongs to the constructor. It's called a **parameter**.

So `this.engine = engine;` means:

```
this.engine
    ↓
Car's field

engine
    ↓
constructor's parameter
```

Therefore `this.engine = engine;` means: "*Take the engine that was given to the constructor and store it inside this `Car`.*"

The whole thing

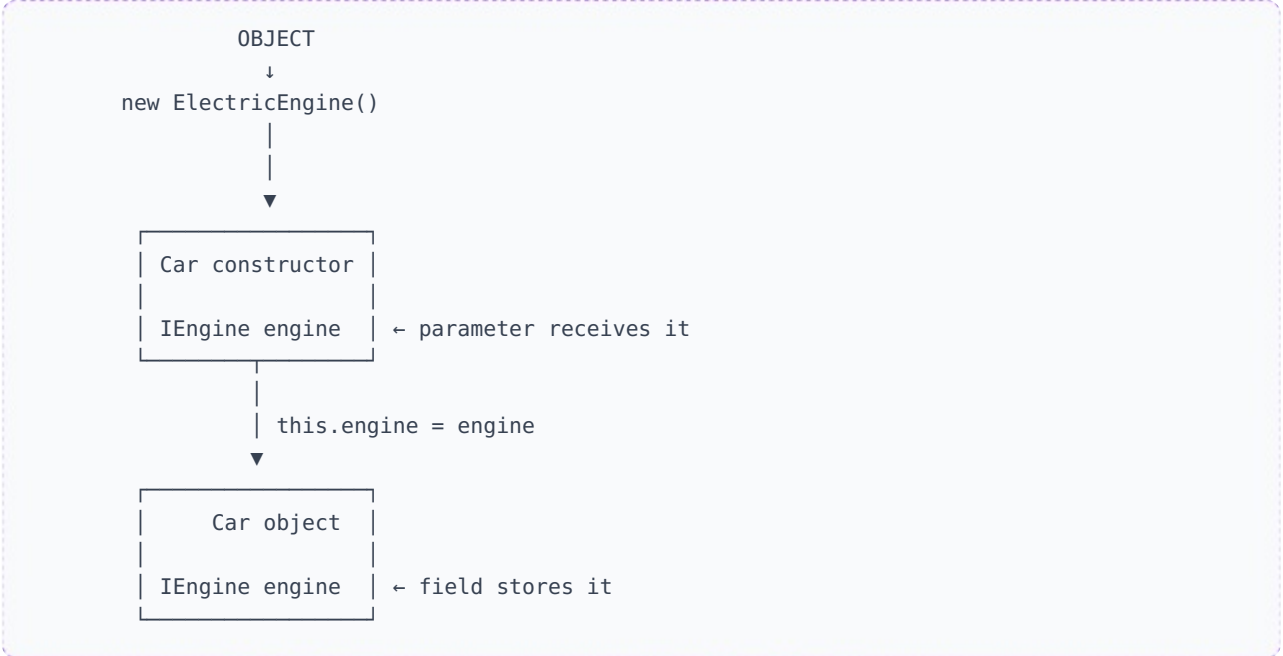
```
interface IEngine
{
    void Start();
}

class ElectricEngine : IEngine
{
    public void Start()
    {
        Console.WriteLine("Electric engine started");
```

```
    }
}

class Car
{
    private IEngine engine;

    public Car(IEngine engine)
    {
        this.engine = engine;
    }

    public void StartCar()
    {
        engine.Start();
    }
}
```

Then:

```
Car car = new Car(new ElectricEngine());
```

Think of it as:



The key idea: `IEngine` in `private IEngine engine;` and `public Car(IEngine engine)` is telling C# that "the reference must be able to refer to an object that implements `IEngine`." It does **not** mean that an interface object itself is being created. The actual object could be `new ElectricEngine()`, or `new GasEngine()`, or `new HybridEngine()`. That's one of the most important ideas behind **polymorphism + dependency injection**.

◆ ◆ ◆

YOU

So I want to know, can:

1. I do it in the constructor to make its parameter just receive any type of class or not?
2. Do dependency injection without an interface, just the normal way?

Yes — both are possible, but there is an important distinction between them.

1. Can the constructor receive "any type of class"?

Not literally any type. You cannot normally write:

```
public Car(anything engine)
```

and have C# accept absolutely any class. But you *can* choose a common type that different classes share.

For example, with an interface:

```
public Car(IEngine engine)
```

This means: "*Give me any object whose class implements* `IEngine`." So these can all be passed:

```
new Car(new ElectricEngine());
new Car(new GasEngine());
new Car(new HybridEngine());
```

because:

```
ElectricEngine ──implements──> IEngine
GasEngine      ──implements──> IEngine
HybridEngine   ──implements──> IEngine
```

You can also use a base class. For example:

```
class Engine
{
    public virtual void Start()
    {
        Console.WriteLine("Engine started");
    }
}

class ElectricEngine : Engine
{
}

class GasEngine : Engine
{
}
```

Then:

```
class Car
{
```

```
    private Engine engine;

    public Car(Engine engine)
    {
        this.engine = engine;
    }
}
```

Now the constructor accepts any class derived from `Engine`:

```
Car car1 = new Car(new ElectricEngine());
Car car2 = new Car(new GasEngine());
```

So the general rule is:

> **The parameter's type determines what kinds of objects are allowed to be passed.**

2. Can I do Dependency Injection WITHOUT an interface?

Absolutely YES. This is very important:

> Dependency Injection does **NOT** require an interface.

DI simply means: *instead of the `Car` creating its dependency itself, somebody gives the dependency to the `Car`.*

Without DI

```
class Car
{
    private ElectricEngine engine;

    public Car()
    {
        engine = new ElectricEngine();
    }
}
```

`Car` is creating its own engine. So:

```
Car
 │
 └── creates → ElectricEngine
```

This creates **stronger coupling**.

Constructor Injection without an interface

You can do:

```
class Car
{
    private ElectricEngine engine;
```

```
    public Car(ElectricEngine engine)
    {
        this.engine = engine;
    }
}
```

Then:

```
ElectricEngine engine = new ElectricEngine();

Car car = new Car(engine);
```

This **IS** Dependency Injection. You injected an `ElectricEngine` object into the `Car`. There is **no interface** involved.

Then why do we use interfaces?

Because an interface makes the dependency more flexible. Compare these two.

Without interface

```
class Car
{
    private ElectricEngine engine;

    public Car(ElectricEngine engine)
    {
        this.engine = engine;
    }
}
```

`Car` is specifically dependent on `ElectricEngine`. You cannot simply give it a `GasEngine`.

With interface

```
class Car
{
    private IEngine engine;

    public Car(IEngine engine)
    {
        this.engine = engine;
    }
}
```

Now `Car` depends on:

So:

```
Car car1 = new Car(new ElectricEngine());
Car car2 = new Car(new GasEngine());
Car car3 = new Car(new HybridEngine());
```

The `Car` doesn't care which implementation it receives.

The most important distinction

Don't think:

> DI = interface — **that's incorrect.**

Think:

> DI = giving an object its dependency from outside instead of having it create the dependency itself.

An interface is one technique that makes DI more flexible. So you have:

```
DI WITHOUT interface
        ↓
Car receives ElectricEngine
        ↓
Still Dependency Injection
        ↓
But tightly coupled to ElectricEngine
```

versus:

```
DI WITH interface
        ↓
Car receives IEngine
        ↓
Can receive ElectricEngine
Can receive GasEngine
Can receive HybridEngine
        ↓
Much looser coupling
```

> **In one sentence:** Dependency Injection is about **HOW** the dependency is supplied; interfaces are about **WHAT** contract the dependency is supplied through.